ROBIN code review

Date: 2026-10-16
Code reviewed: `main` at `493f53f` (LooseLab/ROBIN, forked to gmc-norr/ROBIN)
Scope: all of `src/robin` (91 Python files, about 82,000 lines), plus packaging, resources and CI config.
Prepared with: Claude Code (an AI-assisted review). Treat each finding as a lead to check, not as a final verdict.
Updated: 2026-10-16, after a second review (Codex): the CNV-1 and GUI-1 fix sizes are corrected, the status lines are brought up to date, and the reproduction scripts are added.

How this was done, and its limits

- **Reviewers.** Seven reviewers worked in parallel, one per area:
  - workflow and job handling
  - CLI, launcher and security
  - GUI screens
  - methylation and classifiers
  - CNV, fusion, target and SNP
  - PDF reports
  - packaging and design
- **Evidence.** Every finding was checked against the code, including whether anything actually calls it. Many were reproduced with small scripts on synthetic data, or by running the real ROBIN functions with Ray and NiceGUI stubbed out.
- **My own spot-checks.** I re-read about ten of the most serious findings myself; all of them held up. They are marked *(spot-checked)*.
- **Not done:**
  - Nothing was run on real BAMs.
  - Ray, NiceGUI, pysam, torch and cnv_from_bam were not installed.
  - The `nanoDX` and `hv_rapidCNS2` submodules were not checked out, so the R and NN code inside them was not reviewed.
- **Confidence labels:**
  - **Confirmed:** traced end to end, or reproduced.
  - **Plausible:** a strong reading of the code, but not fully traced (usually because it depends on timing or on Ray scheduling).

The reproduction scripts are in `2026-10-16-repro/`, with a table of which script shows which finding. They are scripts, not regression tests yet. A "Confirmed" label only becomes repeatable evidence once its script is turned into a test in `tests/`.

Read this first

The pipeline has a sound shape: a file watcher feeds a job graph, each sample keeps its state on disk, and the analyses are pluggable. The main problems are not crashes. **Most of the serious problems give wrong or stale results with no warning.** Three patterns cause most of them:

1. **The methylation numbers fed to the classifiers are biased.** The merge averages percentages without weighting by coverage. matkit counts failed calls in the denominator. NanoDX throws away minus-strand calls.

2. **State lives in places that go stale.** There are caches inside long-lived Ray workers, the CNV control reference gets modified by every patient, the GUI keeps per-page state on a shared object, and there is one "waiting" slot per classifier for all samples.
3. **Errors are swallowed at every layer.** Failed jobs are counted as "completed", and error details are never sent to the GUI. This is why the "Error processing file" log line never showed up in the frontend.

Only two tests exist (added after this review), and no CI runs them.

Fix first

#	Problem	What goes wrong	Where	Size of fix
1	Methylation % is averaged without weighting by coverage	Every classifier gets biased input, and adding depth doesn't make it better	`analysis/temp_utilities.py:285-391`	Small
2	CNV control reference is modified by every patient's reads	Real gains and losses fade over a run; false events appear in the next patient	`analysis/cnv_analysis.py:118-145`	Medium (build the control profile without patient reads)
3	One waiting slot per classifier type, shared by all samples	In multi-sample runs, a sample's final classification can silently never run	`workflow_ray.py:2462-2535`	Medium
4	Classification panel freezes if any classifier is disabled	GUI keeps showing the first result for the rest of the run	`gui/components/classification.py:1003-1021`	Small (two loops)
5	Refresh timers are switched off on any brief reconnect	An open page shows hours-old results, with no sign that they are stale	8 GUI components	Small
6	Ray dashboard open on all network interfaces, with no password	Anyone on the network can run code as the ROBIN user	`cli.py:1725-1731`, `2435-2449`	Small (two places)
7	60-second idle timeout (a test value) left in	A sample is marked Complete and finalised too early; SNP calls cover only part of the data	`gui_launcher.py:420`	One line
8	matkit counts failed calls in coverage; NanoDX drops minus-strand calls	Methylation pushed down; about half the reads never reach NanoDX	`utilities/matkit.py`, `nanodx_analysis.py`	Medium
9	ClinVar handling	IDH1 R132H-type hotspot alleles lost; "conflicting" calls shown as pathogenic; a stale index silently returns nothing	`lightweight_gene_analysis.py`, `snp_processing.py`, `clinvar_manager.py`, `reporting/sections/variants.py`	Small to medium
10	Failed jobs counted as "completed"; errors never sent to the GUI	Stale or empty results look current	`workflow_ray.py`, the handlers	Medium
11	Same BAM can be processed twice, for example after a restart	Coverage and counts doubled	`workflow_ray.py:4949-4977`	Medium
12	PDF report uses the wrong Random Forest cut-offs	The PDF can say "High" where the GUI says "Medium"	`reporting/sections/classification.py:399`	One line

Before trusting more runs: compare ROBIN's merged parquet and classifier inputs with `modkit pileup --combine-strands` on one small real BAM. Make that comparison a regression test.

Upstream: #6 (Ray dashboard) and the MNP-Flex TLS issue (SEC-2) are security problems in upstream ROBIN. Report them to LooseLab privately, not in a public issue.

End-to-end data flow (for orientation)

1. `robin workflow` (`cli.py`) checks the model files, then starts `workflow_ray.run()` (the default) or `workflow_simple` (`--no-use-ray`), plus the NiceGUI app.
2. `RayFileWatcher` finds new BAMs. The `Coordinator` actor batches them per sample (`SampleJobBatcher`, `BATCH_CONFIG`) and dispatches Ray tasks. The job graph is `TRIGGERS`.
3. **preprocessing** (`bam_preprocessor`): reads the sample ID and read counts, and writes `master.csv`. It then fans out to bed_conversion, mgmt, cnv, target and fusion.
4. **bed_conversion**: `matkit.run_matkit` writes a per-BAM CpG parquet. `temp_utilities.merge_modkit_files` filters it to `parquet_filter.txt` and merges it into `<work>/<sample>/<sample>.parquet`.
5. **Classifiers**, triggered by bed_conversion:
   - Sturgeon (ONNX)
   - NanoDX and PanNanoDX (NN pickle)
   - Random forest (an R script in `hv_rapidCNS2`)

   Their input comes from `merge_bedmethyl.load_modkit_data`.
6. **Per-sample analyses:**
   - MGMT (methylartist)
   - CNV (`cnv_from_bam` against `HG01280_control_new.pkl`, with ruptures)
   - target coverage
   - fusion (`fusion_work`, clustering of supplementary reads)
7. target → `igv_bam` → `snp_analysis`, which the GUI triggers. It runs ClairS-TO in Docker (`:latest`), then SnpEff/SnpSift with ClinVar.
8. All state is CSV, parquet, pickle or npy files under `work_dir/<sample>/`. There is no database.
9. The GUI (`gui_launcher.GUILauncher`, `gui/components/*`) polls and unpickles those files. Optionally, MNP-Flex uploads to mnp-flex.org.
10. PDF reports (`reporting/report.create_pdf`) are built from the same sample folder.

---

1. Methylation numbers fed to the classifiers

> **METH-1** CRITICAL · **Confirmed (spot-checked)**: Cumulative `percent_modified` is a mean of means.

- **Where:** `analysis/temp_utilities.py:380-391`
- **What's wrong:** Each merge does `group_by(chrom, chromStart)`. It sums `valid_cov`, `n_mod` and `n_canonical` correctly, but sets `percent_modified = mean(...)` of the old aggregate and the new batch. So the newest batch always carries at least 50% of the weight. The summed counts sit in the same row but are never used.
- **Who reads it:** every classifier and MNP-Flex read only `percent_modified`. `merge_bedmethyl.collapse_minimal_bedmethyl` then takes the mean again, despite a comment there saying "weighted average".
- **Reproduced:**

    - 50 reads at 80%, then 1 read at 0%: stored 40.0%, true 78.4%.
    - Simulation of 500 CpGs at 80%, arriving at 20× depth one read per batch: NanoDX "methylated" calls (≥60%) fell to 81% of sites, against 99% with weighting.
- **Fix:** after summing, set `percent_modified = 100 * n_mod / valid_cov`.

METH-2 HIGH · **Confirmed**: The first write isn't aggregated, so random forest can see values over 100%.

- **Where:** `temp_utilities.py:285-333`
- **What's wrong:** When no cumulative parquet exists yet, the concatenated per-BAM rows are written as they are. A first batch can hold up to 20 BAMs, so there is one row per BAM per site.
- **What goes wrong downstream:**
    - The random forest path sums `score`, so the R script gets methylation of 200-300%.
    - Sturgeon counts the duplicate rows as separate sites.
- **Fix:** always run the same aggregation before writing (this is the same fix as METH-1).

METH-3 HIGH · **Confirmed**: matkit puts failed calls into `valid_cov`.

- **Where:** `utilities/matkit.py:1344, 1373-1385, 1132-1139`
- **What's wrong:** The total is incremented for every read before it is classified, so fails are counted. The result is `n_mod / (mod + canonical + fail)`; modkit leaves fails out of the denominator.
- **Example:** 6 modified, 2 canonical and 2 failed reads are stored as 60% instead of 75%. That moves the site across Sturgeon's 0.7 cut-off and onto NanoDX's 60% cut-off.
- **Plausible, secondary:** with combine_mods, the call uses max(P_m, P_h, P_C) rather than comparing P_m+P_h with P_C. This matters in brain tissue, which has high 5hmC.

METH-4 HIGH · **Confirmed**: NanoDX and PanNanoDX drop every minus-strand CpG call.

- **Where:** `nanodx_analysis.py:342-349`, `utilities/merge_bedmethyl.py:490-501`
- **What's wrong:** matkit records the minus-strand call at C+1. The collapse step relabels the strand as "." but still groups on position, so the + and − rows of one CpG never merge. NanoDX's exact join on the 0-based C position then keeps only the + row.
- **Effect:**
    - About half the reads never reach NanoDX.
    - Sturgeon gets the two strand rows as equal votes, ignoring coverage: 3 reads at 100% count the same as 20 reads at 0%.
- **Fix:** shift − rows to position−1, then sum counts per CpG (the equivalent of `--combine-strands`).

METH-5 MEDIUM · **Confirmed**: Panel intersect plus inner join double-counts sites.

- **Where:** `temp_utilities.py:250-266`
- **What's wrong:** `parquet_filter.txt` has 976 pairs of overlapping intervals, including 90 exact duplicates. After the widening in METH-6 there are 8,238. PyRanges `intersect` returns one row per overlap, so the join duplicates the site.
- **Example:** in a CGCG, coverage is stored as 16 instead of 8.
- **Fix:** use a semi-join, and remove duplicates from the panel.

METH-6 MEDIUM · **Confirmed (spot-checked)**: `parquet_filter.txt` is already 0-based, so the v0.5 "1-based fix" is wrong.

- **Where:** `temp_utilities.py:159-179`, `docs/releases/v0.5.md:19`

- **Evidence:** cg13869341 is at `15864-15866` in this file, in the 0-based NanoDX BED and in the 0-based MNP-Flex BED. About 486k of its probes match the NanoDX file exactly.
- **What's wrong:** `Start - 1` widens every CpG to 3 bp, and the release note wrongly tells users their old parquets were wrong.
- **Also:** the first data row is `chr1 -1 0 cg22940546`, which has a negative start.
- **Fix:** read the file as plain 0-based BED, and correct the release note.

METH-7 MEDIUM · **Confirmed**: Classifiers never see coverage, and random forest gets a constant `cov=10`.

- **Where:** `merge_bedmethyl.py:72-189, 751-797`, `random_forest_analysis.py:283-346`
- **What's wrong:** The loader keeps only 5 columns. The random forest input invents `valid_cov=10`. So a site with 1 read counts the same as a site with 50.

METH-8 MEDIUM · **Confirmed**: Classifier failures are reported as success.

- **Where:** `sturgeon_analysis.py:585-588`, `merge_bedmethyl.py:93-102`, `nanodx_analysis.py:186-190, 366-397`, `mgmt_analysis.py:842-848`
- **Per classifier:**
  - **Sturgeon:** catches and logs the error, and appends no row.
  - **NanoDX:** appends an empty row with `number_probes=0`, and writes patient data to `errordf.csv` in the worker's current directory.
  - **MGMT:** an R failure only logs a warning, and the step returns True.
- **Effect:** the GUI keeps showing the previous result as if it were the latest.

METH-9 LOW/MEDIUM · **Confirmed (mechanism)**: One BAM failing in a batch is skipped and its data deleted.

- **Where:** `temp_utilities.py:268-270`, `bed_conversion.py:252-262, 323-329`
- **What's wrong:** Per-file errors in the merge are logged and the loop goes on. All the per-BAM parquets are then deleted, including the failed one, and the job reports success.
- **Context:** this is the path behind the original "Error processing file ... ['Chromosome', 'Start']" message. That case is fixed on `main` (commit `d02add2`), and it lost no data. Commit `5be9e04` makes this log line include a traceback and safe diagnostics.

METH-10 LOW · **Confirmed**: The MGMT site table can show 0.43% as 43%.

- **Where:** `mgmt_analysis.py:336-347`; the GUI copy is at `gui/components/mgmt.py:226-235`
- **What's wrong:** It guesses "fraction or percent?" from whether any row is above 1. An unmethylated promoter at high depth fools the guess. The GUI copy has also drifted from the analysis copy.

METH-11 LOW · **Plausible**: The `bam_file_count` metadata undercounts.

- **Where:** `bed_conversion.py:300`, `temp_utilities.py:275`
- **What's wrong:** bed_conversion always passes `num_bam_files_seen=1`, and the count isn't updated when every file in a batch is skipped.

METH-12 LOW · **Plausible**: The panel filter cache is keyed by the panel file's basename only.

- **Where:** `temp_utilities.py:152`
- **What's wrong:** A different panel file with the same name reuses the old ranges. The cache pickle is also written non-atomically, outside the lock.

Not a problem:

- The 5hmC+5mC merge is by design.
- MGMT's 1-based to 0-based site handling is correct.
- Parquet writes are atomic.
- The cumulative merge is bounded by the panel size, not by total reads.

---

2. CNV

> **CNV-1** CRITICAL · **Confirmed (spot-checked)**: The cached control reference is modified in place and absorbs every patient's reads.

- **Where:** `analysis/cnv_analysis.py:118-145, 1181-1217, 1491, 1535-1597`
- **What's wrong:** `get_cached_ref_cnv_dict` returns one module-global dict, and it is passed straight to `cnv_from_bam.iterate_bam_file(copy_numbers=...)`, which adds reads into it (checked against the upstream Rust source). So in a long-lived Ray worker, the "control" grows with every BAM of every sample.
- **Simulated effect:**
  - Within one patient, a 3-copy gain reads +1.04 after the first batch, +0.49 when the patient has as many reads as the control, and +0.28 at 2.5× the control. A partial-purity gain drops below the ±0.4 call threshold, so events like 1p/19q or +7/−10 can disappear as the run goes on.
  - Across patients, a normal patient processed after another one showed mirror-image false events.
- **Fix:** build the reference profile from the stored control counts only, by re-binning them to the sample's bin width in numpy. Never pass a dict that must stay clean to `iterate_bam_file`.
- **Not enough on its own:** a `copy.deepcopy` per call is only a stopgap. It stops patients polluting each other, but each call still adds the current BAM to the copy, so the reference still contains that BAM's reads.

> **CNV-2** HIGH · **Plausible**: Per-sample CNV counts are cached per Ray worker and never invalidated.

- **Where:** `cnv_analysis.py:1137-1178, 1446-1484, 1245, 1620`, `workflow_ray.py:392-394, 2037-2068`
- **What's wrong:**
  - The cache only checks that the path matches.
  - CNV isn't serialised per sample.
  - The pickle is written non-atomically, and a load error silently resets the counts to `{}`.
  - The Coordinator's cache cleanup runs in the wrong process, so it does nothing.
- **Example:** batch 1 runs on worker A, batch 2 on worker B, and batch 3 on worker A again. Worker A uses its stale cache, so batch 2's reads are lost for good.
- **Fix:** always reload from disk under a per-sample lock, and write with tmp + `os.replace`.

> **CNV-3** MEDIUM · **Confirmed**: Multi-BAM CNV adds the batch's last BAM twice.

- **Where:** `cnv_analysis.py:1515-1597, 1620`
- **What's wrong:** The loop has already accumulated every BAM, and the "final aggregated" call then adds the last one again. The result is saved, so the double count is permanent.

> **CNV-4** MEDIUM · **Confirmed**: The sex labels don't match the thresholds.

- **Where:** `cnv_analysis.py:677-689`, `classification_config.py:164`, `cnv_classification.py:195`, `gui/components/cnv.py:751-760`, `reporting/sections/cnv.py:365-373`
- **What's wrong:**
  - The estimator emits "Male (query X/Y ...)", which the threshold code treats as female.
  - The report compares against "XX"/"XY", which are never produced.

- **Effect:** for such males, a chrX gain needs >0.75 instead of >0.3, and chrY isn't skipped.
- **Fix:** use one shared XX/XY/UNKNOWN enum.

> **CNV-5** LOW · **Confirmed (by reading)**: Every chromosome gets a fake breakpoint at its end.

- **Where:** `cnv_analysis.py:574-580, 714-730`
- **What's wrong:** ruptures always returns the series length as its last change point. That adds about 24 spurious regions to the master BED.
- **Fix:** use `result[:-1]`.

The CNV summary card and the report's CNV problems are covered in sections 6 and 7.

---

3. Variants, ClinVar, fusions, custom panels

> **VAR-1** HIGH · **Confirmed**: ClinVar records at the same position overwrite each other.

- **Where:** `analysis/lightweight_gene_analysis.py:228-229, 560-612`
- **What's wrong:** The lookup is keyed by `(chrom, pos)`, so only the last ALT allele at a position is kept.
- **Reproduced:** with IDH1 R132H (C>T) followed by C>A in the file, a patient who carries R132H shows "no_support". BRAF V600 and KRAS G12-type sites have the same problem.
- **Fix:** key by `(chrom, pos, ref, alt)`.

> **VAR-2** HIGH · **Confirmed**: "Conflicting classifications of pathogenicity" counts as pathogenic.

- **Where:** `snp_processing.py:14-23, 132-136`, `lightweight_gene_analysis.py:260-265`, `reporting/sections/variants.py:61-75`
- **What's wrong:**
  - Substring matching on "pathogenic" also matches "Conflicting_..._of_pathogenicity". In the report, the CLNSIGCONF vote branch is therefore never reached, and it would crash if it were.
  - Also: `parse_vcf` groups with `dropna=True`, so records without ANN vanish from the table. In the test, 2 of 3 variants were shown.
- **Fix:** match exact CLNSIG tokens, use one shared helper, and `groupby(..., dropna=False)`.

> **VAR-3** HIGH · **Confirmed (reproduced with bcftools)**: A ClinVar refresh keeps the old tabix index.

- **Where:** `utils/clinvar_manager.py:130-131, 221, 300`
- **What's wrong:** After a new `clinvar.vcf.gz` is swapped in, `_ensure_tabix_index` returns early because the old `.tbi` exists. Region queries then return nothing with exit code 0, so pathogenic annotations silently disappear.
- **Fix:** always rebuild the index after replacing the file.

> **VAR-4** HIGH · **Confirmed**: The report says "No pathogenic variants" when ClinVar annotation never ran.

- **Where:** `reporting/sections/variants.py:229-246, 306-335`, `target_analysis.py:3891-3940`
- **What's wrong:**
  - If the ClinVar DB is missing or SnpSift fails, plain snpEff output is copied to `snpsift_output.vcf`, and the report reads that as "annotated, none found".
  - The file is also truncated before each re-annotation.
- **Fix:** write atomically, store an annotation-status marker, and have the report say "ClinVar annotation unavailable".

VAR-5 MEDIUM · **Confirmed**: Failed ClairS-TO regions are skipped silently.

- **Where:** `target_analysis.py:3549-3704`
- **What's wrong:**
  - A container that fails (including an OOM kill, exit 137) only logs a warning, and the merge reports "completed successfully for all regions".
  - Failed containers are never removed.
  - Docker and snpEff have no timeouts.

VAR-6 LOW · **Confirmed**: VCF→CSV uses the first variant as the header row.

- **Where:** `target_analysis.py:4143-4165`, `gui/components/coverage.py:3913-3933`
- **Effect:** the SNP/INDEL counts are one low, and a VCF with a single variant shows 0.

VAR-7 HIGH · **Confirmed**: `robin add-panel` keeps only the first region of each gene.

- **Where:** `cli.py:957`
- **What's wrong:** `drop_duplicates(subset=['gene'])` drops every later exon or tile of a gene. In the test, a panel with 3 EGFR rows and 2 TP53 rows became 2 rows. Coverage, SNP calling and fusion detection then ignore the dropped regions.

FUS-1 MEDIUM · **Confirmed (simulated)**: Master-BED SV breakpoints almost never reach the support threshold.

- **Where:** `fusion_work.py:1670-1672, 1775-1855`
- **What's wrong:** Reads are keyed by the midpoint of the whole alignment. The supplementary span is taken as `pos + query_length`, which is 1-based and covers the whole read. Supplementary records also pair with themselves.
- **Simulated:** 5 to 50 reads through one junction never put 3 or more reads in the same key, so zero breakpoints were written.

FUS-2 MEDIUM · **Confirmed**: The breakpoint BED is keyed to the CNV counter, so earlier breakpoints are dropped.

- **Where:** `fusion_work.py:3072-3090, 3879-3913, 4265-4272`, `master_bed_generator.py:85-115, 1073`
- **What's wrong:** Fusion reads `master_bed_breakpoints_{cnv_counter}.bed`, and the counter moves on almost every batch. So each new file holds only the new breakpoints, and earlier SV regions fall out of the dynamic targets.

FUS-3 MEDIUM · **Confirmed (deletion); Plausible (double append)**: Fusion accumulation has no lock and deletes staging files that failed to load.

- **Where:** `fusion_work.py:2175-2437`
- **What's wrong:** That BAM's fusion reads are lost. With `analysis_workers > 1`, supporting reads can be counted twice.

---

4. Scheduling and job handling

JOB-1 CRITICAL · **Confirmed (spot-checked)**: One global waiting slot per classifier type.

- **Where:** `workflow_ray.py:998-1000, 2462-2535, 2807-2871`

- **What's wrong:** Classification is limited to one running job per type, plus one waiting job per type for all samples. If the slot is taken, a new request is dropped with no else branch.
- **Reproduced:** with samples A and B, Sturgeon ran for A twice and never for B.
- **Fix:** keep a "dirty" flag per (sample, type), and run dirty samples round-robin.

> **JOB-2** HIGH · **Confirmed**: The same BAM can be processed twice.

- **Where:** `workflow_ray.py:4949-4977, 4116, 4155-4160, 4510-4551`
- **What's wrong:** The start-up scan doesn't add files to the watcher's dedup set. `add_watch_path` starts watching before it scans. There is no per-file record across restarts (`SQLiteStateTracker` exists but has no callers). Methylation counts, CNV and `master.csv` totals all just add up.
- **Effect:** a restart with the same `--work-dir` counts everything twice.
- **Fix:** keep a persistent per-file ledger, and share one dedup set.

> **JOB-3** HIGH · **Confirmed**: A failed handler runs again without `work_dir`, `reference` or `target_panel`.

- **Where:** `workflow_ray.py:704-742`
- **What's wrong:** The `try` wraps the handler call itself, so any exception falls through to the legacy `py_handler(job)`. The handler then runs a second time, writes output next to the BAM (inside the MinKNOW folder), and the real error is replaced by "target_panel is required".
- **Fix:** bind the arguments first, then call the handler once.

> **JOB-4** HIGH · **Confirmed (flush and ordering); Plausible (partial files)**: The watcher delays or loses files.

- **Where:** `workflow_ray.py:4117-4193, 5020-5030`
- **What's wrong:**
  - The pending buffer only flushes when a new file arrives, and there is no timer, so the last file(s) of a run may never be analysed.
  - The flush at shutdown happens after the coordinator has stopped.
  - Submit errors are swallowed after the file has been marked processed.
  - There is no check that a file is complete, so a BAM copied in with `cp` or `scp` can be read truncated.

> **JOB-5** HIGH · **Confirmed**: Repeat SNP requests are silently skipped while the UI says "queued".

- **Where:** `workflow_ray.py:1730-1737, 2991-2997, 2363-2367`
- **What's wrong:** The dedup counter is never decremented (`mark_running` has no callers), so after the first `snp_analysis` for a sample, every rerun is dropped.

> **JOB-6** HIGH · **Constant confirmed (spot-checked); impact Plausible**: The 60-second idle timeout is a test value.

- **Where:** `gui_launcher.py:420, 927-934, 7406, 7424-7426`, `target_analysis.py:2604-2618`
- **What's wrong:**
  - The code still says `completion_timeout_seconds = 60  # Set to 3600 for production`.
  - A gap of 60 s or more between BAMs marks the sample Complete and finalises it. Later SNP runs then skip the merge and use the old `target.bam`.
  - After a restart, unmerged batch BAMs are deleted.

> **JOB-7** HIGH · **Double run Confirmed; corruption Plausible**: `target.bam` finalisation runs twice.

- **Where:** `gui_launcher.py:7422-7512`

- **What's wrong:** `finalization_succeeded` stays True after the Ray job is queued, so the GUI also runs the finalisation inline. The two copies delete each other's temp and batch files.
- **Fix:** run the inline finalisation only when the submit fails, and add a per-sample lock.

> **JOB-8** MEDIUM · **Confirmed**: Files submitted by the watcher have no `reference` in their metadata.

- **Where:** `workflow_ray.py:4155-4185` vs `3756-3771`
- **What's wrong:** Files found at start-up get the reference and target panel; files that arrive later don't. So mgmt, target, CNV and fusion behave differently depending on when a BAM appeared.

> **JOB-9** MEDIUM · **Confirmed (opt-in setting)**: A job timeout releases its pool slot twice.

- **Where:** `workflow_ray.py:3575-3599` vs `3527-3534`
- **What's wrong:** With `ROBIN_JOB_TIMEOUT_SECONDS` set, each timeout adds a phantom slot, so a pool limited to 1 job runs jobs at the same time.

> **JOB-10** MEDIUM · **Plausible**: Shutdown kills jobs mid-write and drops waiting jobs.

- **Where:** `workflow_ray.py:3281-3363, 5020-5054`
- **What's wrong:** A Ctrl-C during a CNV save leaves a truncated pickle, and the next start silently resets the CNV history.

> **JOB-11** MEDIUM · **Confirmed**: The default Ray path ignores several CLI options.

- **Where:** `cli.py:2419-2537`
- **What's wrong:** `--no-gui`, `--commands`, `--deduplicate-jobs`, `--queue-priority` and `--show-priorities` have no effect. For example, `--show-priorities` starts a full run, and `--no-gui` still opens the GUI.

> **JOB-12** LOW · **Plausible**: Race conditions when `analysis_workers > 1`.

- **What's wrong:** MGMT's `mgmt.bam` accumulation and file numbering have no lock, and neither does the panel filter cache pickle. The default of 1 worker avoids this.

> **JOB-13** LOW · **Confirmed (latent)**: All job types for a file share one context object.

- **Where:** `workflow_ray.py:2554, 2947-2964`
- **What's wrong:** `_batched_job` is overwritten by each type's batch. This is harmless today only because every type's batch holds the same files.

---

5. Errors that never reach the user

This is why "Error processing file" showed up in the backend log but never in the GUI.

> **ERR-1** HIGH · **Confirmed**: Errors are lost at five layers.

- **Where:** `workflow_ray.py:3515-3521, 801-809, 4733-4928, 755`
- **Layer by layer:**
  1. The merge swallows per-file errors and reports success (METH-9).
  2. Handlers record errors under `"<type>_analysis"`, but the Pool only checks the `job_type` key, and the `finally` block adds `{"status": "success"}`. So **for 8 of 13 job types, a failed job counts as completed**: it isn't in `failed_jobs.csv` or in the failed count.

3. `_publish_gui` sends only counts. Nothing ever sends `ERROR_UPDATE`, even though the GUI has handlers for it.
4. `send_gui_update` calls from inside Ray workers find no GUI in that process and are dropped. This includes BAM alignment warnings such as "unaligned BAM" or "<1% mapped".
5. The TypeProcessor path (preset `high`) has no error detection at all.

- **Fix:** treat any `add_error` during a job as a failure, return per-file failures from the merge, forward failures from the driver to the GUI, and route worker notifications through the Coordinator.

> **ERR-2** MEDIUM · **Confirmed**: Import failures are swallowed and reported with the wrong cause.

- **Where:** `analysis/__init__.py:15-97`, `cli.py:55-81`, `workflow_ray.py:124-200`, `nanodx_analysis.py:24-38`, `temp_utilities.py:30-34`
- **Example:** a Sturgeon API change makes every Sturgeon job fail with "No implementation available for job_type='sturgeon'", with no reason given.
- **Missing submodules:** an empty submodule folder isn't detected.
- **Fix:** import lazily per job type, and print every import error at start-up.

> **ERR-3** · **Confirmed**: The codebase has 1,325 broad `except Exception` or bare `except` blocks, and 404 of them do nothing but `pass` or `continue`.

---

6. GUI

> **GUI-1** CRITICAL · **Confirmed (spot-checked)**: The classification panel stops updating when any classifier is disabled.

- **Where:** `gui/components/classification.py:1003-1021`
- **What's wrong:** The loop goes over all four tools in `tool_to_file`, but `charts` only has entries for the enabled ones. So `charts['PanNanoDX']` raises KeyError on every tick after the first. The panel keeps showing the first result, while the summary card moves on.
- **Fix:** in both loops of `_refresh_classification` (lines 1003-1008 and 1019-1021), loop over `charts` instead of `tool_to_file`, or skip tools that aren't in `charts`. The third loop (1041-1047) is already guarded.

> **GUI-2** CRITICAL · **Confirmed (against NiceGUI 3.6 source)**: A brief websocket drop permanently stops every refresh timer.

- **Where:** `summary.py:352`, `classification.py:1068`, `coverage.py:6917`, `mgmt.py:721`, `cnv.py:2467`, `fusion.py:3421`, `mnpflex.py:860`, `bed_coverage.py:456`
- **What's wrong:** Every component calls `timer.deactivate()` in `on_disconnect`, which in NiceGUI 3 also fires on reconnect, and nothing reactivates the timers.
- **Effect:** after a Wi-Fi blip, laptop sleep or proxy timeout, the page shows hours-old results.
- **Fix:** use `client.on_delete`.

> **GUI-3** HIGH · **Confirmed (MGMT simulated)**: A second viewer of the same sample misses updates.

- **Where:** `mgmt.py:388-405`, `cnv.py:1989-2010`, `coverage.py:6497-6531`, `gui_launcher.py:491-499`
- **What's wrong:** The "last seen mtime" markers live on the one shared `GUILauncher`. The first browser to see a new file records it, and the others then think nothing changed.
- **Reproduced:** after a new batch, browser A showed "Methylated" and browser B still showed "Unmethylated".

GUI-4 HIGH · **Confirmed (spot-checked)**: The CNV summary card shows the batch counter as "Gained".

- **Where:** `summary.py:1245-1292`
- **What's wrong:** It reads `cnv_analysis_counter.txt` as the number of gains. After 37 batches the card says "Gained: 37, Lost: 0" for every sample. The other files it looks for are never written.

GUI-5 HIGH · **Confirmed (by reading)**: Per-page state is shared by all browsers.

- **Where:** `gui_launcher.py:2396, 2732, 3155-3163, 7289-7317`
- **What's wrong:** The sample table, selection and filters are stored on the singleton.
- **Example:** user B opening `/live_data` clears user A's selection, so A's Export gives B's samples.

GUI-6 MEDIUM · **Confirmed (path); race Plausible**: The GUI rewrites analysis output files.

- **Where:** `summary.py:1612-1626` → `fusion.py:307-404`, `mgmt.py:594-603`
- **What's wrong:** Every 30 s, for each browser, it rewrites `fusion_summary.csv`, `fusion_results.csv` and `sv_count.txt` non-atomically, using its own counting method. MGMT also overwrites `{N}_mgmt.pkl`.
- **Fix:** make the GUI read-only over sample folders.

GUI-7 MEDIUM · **Confirmed**: The summary card and the coverage panel use different coverage thresholds.

- **Where:** `summary.py:1180-1187` vs `coverage.py:6681-6699`
- **Example:** at 12× the card says "Moderate" while the panel says "Low".

GUI-8 MEDIUM · **Confirmed**: The GUI fusion check uses a different minimum support.

- **What's wrong:** It re-validates with min_support=4, while the analysis and `fusion_summary.csv` use 3. So the summary and the fusion tab can disagree.

GUI-9 LOW · **Confirmed**: The MGMT card's CpG count says "Not available" or is wrong.

- **Where:** `summary.py:1338-1388`
- **What's wrong:** The helper is defined in one branch only, so the other branch hits a NameError.

GUI-10 LOW · **Confirmed**: The "global" theme and metrics timers die with the first client.

- **Where:** `theme.py:127-134, 437-443`
- **What's wrong:** They are bound to the first client. Also, other users' charts follow the first user's dark-mode setting.

---

7. PDF report

REP-1 HIGH · **Confirmed (spot-checked)**: Random Forest confidence uses the default cut-offs.

- **Where:** `reporting/sections/classification.py:399`
- **What's wrong:** It looks up `"random forest"`, but the key is `"random_forest"`, so the default 80/50 is used instead of 85/65. At 82%, the PDF says "High" and the GUI says "Medium".
- **Fix:** map display names to config keys, and raise an error on an unknown classifier.

REP-2 HIGH · **Confirmed**: CNV gene counts and plot selection use an old ±1 SD rule.

- **Where:** `reporting/sections/cnv.py:122-160, 580-620, 988-1005`
- **What's wrong:** The GUI switched to fixed thresholds; the report still uses the old rule.
- **Reproduced:** on a CNV-neutral genome, the report listed 20 "gained" and 17 "lost" genes and plotted all 23 chromosomes, while its own events table was empty.

REP-3 HIGH · **Confirmed**: The fusion table shows the weakest breakpoint cluster.

- **Where:** `reporting/sections/fusion.py:249-262`
- **What's wrong:** The last row wins, and the rows are sorted strongest first. For example, KIAA1549-BRAF shows a 4-read cluster in the PDF, while the CSV export shows 25 reads.

REP-4 HIGH · **Confirmed**: Gene pairs are split on "-", which breaks hyphenated genes.

- **Where:** `reporting/sections/fusion.py:53, 121-127, 254, 561`
- **What's wrong:** Names like `EGFR-AS1` and `H3-3A` get cut in two. For example, EGFR-SEPTIN14 becomes the network "EGFR,EGFR - AS1", and SEPTIN14 disappears.
- **Fix:** carry gene1 and gene2 as separate fields.

REP-5 HIGH · **Plausible**: Report plotting shares global matplotlib state with GUI threads.

- **Where:** `reporting/plotting.py:340-430`, `analysis/methylation_wrapper.py:13-25, 256-293`
- **What's wrong:** The report uses pyplot's global state in a NiceGUI worker thread, while the GUI's MGMT refresh monkeypatches `Figure.savefig` process-wide.
- **Effect:** a plot can be swallowed, or another figure (possibly another sample's) embedded.
- **Fix:** use the object-oriented `Figure` API, or run report generation in a subprocess.

REP-6 MEDIUM · **Confirmed**: Missing fusion data is reported as "No fusion networks identified".

- **Where:** `reporting/sections/fusion.py:133-197, 425-441, 640-645`
- **What's wrong:** "Not run", "unreadable" and "none found" all print the same negative result. The methods text also states different thresholds from the code (>50 vs >49, >200 bp vs >249 bp).

REP-7 MEDIUM · **Confirmed**: CNV plot y-limits hide focal amplifications.

- **Where:** `reporting/plotting.py:264-267, 297, 356-357, 420`
- **What's wrong:** The limit is mean + 2 SD. An EGFR amplification at about 30 copies falls off the axis (the limit is about 8.6).

REP-8 MEDIUM · **Confirmed**: At low CNV resolution, the section crashes after half the summary is built.

- **Where:** `reporting/sections/cnv.py:689-723, 805`
- **What's wrong:** `events` is unbound, so the summary shows the CNV heading with no events and no "resolution insufficient" note.

REP-9 MEDIUM · **Confirmed**: A coverage file being written can abort the whole report.

- **Where:** `reporting/sections/coverage.py:28-31, 55-113`, `report.py:59-67, 142-144`
- **What's wrong:** The coverage CSVs are read in the constructor, outside the per-section error handling.

REP-10 MEDIUM · **Plausible**: The report reads live, non-atomic files with no snapshot.

- **Where:** `reporting/sections/cnv.py:418-433, 492-496`, `reporting/sections/classification.py:339-413`
- **What's wrong:** CNV.npy, CNV_dict.npy and CNV2.npy can come from different iterations. A stale `PanNanoDX_scores.csv` from an earlier configuration can appear in the PDF but not in the GUI.

---

8. Security and privacy

SEC-1 CRITICAL · **Confirmed (spot-checked)**: The Ray dashboard and Jobs API listen on 0.0.0.0 by default, with no authentication.

- **Where:** `cli.py:2435-2449, 1725-1731, 2302-2306`, `workflow_ray.py:5161`
- **What's wrong:** Anyone on the network can POST a job to `http://<host>:8265/api/jobs/` and run shell commands as the ROBIN user, with access to all BAMs and reports. This is the ShadowRay / CVE-2023-48022 class of issue.
- **Fix:** default to `127.0.0.1` and make the dashboard opt-in.

SEC-2 HIGH · **Confirmed**: MNP-Flex TLS verification is hard-coded off.

- **Where:** `gui_launcher.py:7180, 7225-7231`, `gui/components/mnpflex.py:92-147, 829-855`, `utils/mnpflex_client_standalone.py:34`
- **What's wrong:**
  - The credentials (the `client_secret` defaults to `"SECRET"`) and the patient methylation BED are sent without certificate checks, and the downloaded classification is shown as a result.
  - The upload starts automatically, once per open tab.
  - On failure, patient data is left on the third-party server.

SEC-3 HIGH · **Confirmed (reproduced)**: Partial model downloads are kept and accepted.

- **Where:** `utils/model_updater.py:121-152, 290-306`, `utils/model_checker.py:181-187`
- **What's wrong:** Downloads stream straight to the final filename, and the start-up check only tests `size > 0`. The models are pickles, so a swapped file means code execution.
- **Fix:** download to `.part`, verify the SHA-256, rename, and re-verify at start-up.

SEC-4 MEDIUM · **Confirmed**: The GUI listens on 0.0.0.0 over plain HTTP.

- **Where:** `cli.py:2284-2289`, `gui_launcher.py:535-546, 1845-1863, 2122-2131`
- **What's wrong:**
  - `storage_secret="robin"` is a constant.
  - Logins aren't throttled, and the argon2 check runs on the event loop.
  - There is an open redirect via `redirect_to`.
  - The "local client" check trusts the Host header.

SEC-5 MEDIUM · **Mechanism Confirmed; exploit Plausible**: The auth middleware lets some routes through without checks.

- **Where:** `gui_launcher.py:549-553, 2113-2117`
- **What's wrong:**
  - `/_nicegui*` and `/api/v1/*` are exempt, and `ui.download` URLs are predictable.

- `/api/download/{sample_id}/{filename}` accepts `..` (one level up).

SEC-6 MEDIUM · **Confirmed**: Patient identifiers can be decrypted from the date of birth alone.

- **Where:** `gui_launcher.py:98-131, 8788-8806, 4407-4431`, `reporting/report.py:193-204`
- **What's wrong:**
  - The key comes from PBKDF2(DOB, fixed salt). That is about 36,500 possible keys, which takes about 20 s on a 72-core server.
  - The sample ID is an unsalted MD5, and it is sent to MNP-Flex.
  - PDFs with plaintext identifiers are left in the sample folder.

SEC-7 MEDIUM · **Plausible**: About 30 sites load pickles from results folders.

- **Examples:** `gui/components/summary.py:1221`, `reporting/sections/cnv.py:433`, `fusion_work.py:2601`
- **What's wrong:** Opening a results folder, or building a PDF from one, received from someone else can run code.

---

9. Packaging, dependencies, tests

PKG-1 HIGH · **Confirmed**: Builds aren't reproducible.

- **Where:** `pyproject.toml:38-40, 48, 55`, `target_analysis.py:3274, 3581`
- **What's wrong:**
  - `sturgeon@…@testing` tracks a moving branch.
  - `methylartist` has no git ref at all.
  - torch, polars, pyranges and ray are unpinned, and there is no lockfile.
  - ClairS-TO is pulled as `:latest`.
  - The GUI's CrossNN version lookup uses a path that doesn't exist, so it always shows "unknown".
- **Effect:** a report can't say which classifier or caller code produced it.

PKG-2 MEDIUM · **Confirmed**: There are no tests and no CI for code on `main`.

- **What's wrong:**
  - pytest's `tests/` folder holds only the two tests added after this review (`tests/test_temp_utilities.py`).
  - CI only builds the docs, on Python 3.11.
  - The mypy and black configs still target Python 3.8.

PKG-3 MEDIUM · **Confirmed**: Python 3.13+ installs fail.

- **Where:** `pyproject.toml:21`
- **What's wrong:** `requires-python = ">=3.12"`, but numpy 1.26, pandas 2.1.4 and pyarrow 16.1 have wheels only up to 3.12, and sturgeon@testing requires ≤3.12.
- **Fix:** add `<3.13`.

PKG-4 MEDIUM · **Confirmed**: Unpinned polars and pyranges are already causing problems.

- **What's wrong:**
  - `pl.StringCache` gives a deprecation warning on polars 2.0 (`temp_utilities.py:283`).
  - `ncls` is imported but not declared (`fusion_work.py:43`).

- PyRanges objects are pickled into the sample folders.

> **PKG-5** MEDIUM · **Confirmed**: Resource files and the gitignore need cleaning up.

- **What's wrong:**
  - About 113 MB of unused or duplicate resources ship in every wheel. Examples: a 45 MB gencode GFF, two 20.6 MB MNP-Flex BED variants, and `None_data.csv.gz`. Some are byte-identical copies.
  - `.gitignore` misses the v2 model files (340 MB).
  - `src/robin/analysis/Untitled` is tracked in git.

> **PKG-6** LOW · **Confirmed**: The dependency list and licence need tidying.

- **What's wrong:**
  - 10 dependencies are listed twice, about 13 are never imported, and 6 that are imported aren't declared.
  - `robin.yml` leaves numpy unpinned while pip pins `<2`.
  - The package is labelled MIT but bundles GPL-3.0 nanoDx. That one is a licence question to raise.

---

10. Architecture

- **ARCH-1: Two workflow engines that have drifted apart.**
  - `workflow_ray.py` (5.2k lines) and `workflow_simple.py` each define their own `Job`, `WorkflowContext`, batcher and watcher.
  - Their `BATCH_CONFIG` differs (CNV 20 vs 50).
  - Inside workflow_ray there are three dispatch paths, each with different error detection.
  - Per-sample state is spread over about 15 dicts.
  - JOB-2, JOB-8 and ERR-1(5) all come from this.
- **ARCH-2: God files and god functions.**
  - `gui_launcher.py` is 9.2k lines.
  - `add_coverage_section` is one 5,181-line function, `add_cnv_section` is 2,260 lines and `run_snp_analysis` is 1,233 lines.
  - `fusion_work.py` is 6.5k lines.
  - None of these can be unit-tested.
- **ARCH-3: The same logic is copied into the analysis, GUI and report, and the copies have drifted.** Examples: cytoband CNV calls, fusion clustering and validation, pathogenicity checks, confidence and coverage thresholds, MGMT site parsing. REP-1, REP-2, GUI-7, GUI-8 and METH-10 are the results.
- **ARCH-4: Lots of dead code that still gets edited.** Examples:
  - matkit's copy of `merge_modkit_files`
  - about 1,000 lines in `fusion_work`
  - about 980 lines under `if False:` in `coverage.py`
  - about 800 lines in `gui_launcher`
  - 7 of 9 functions in `reporting/plotting.py`
  - `reporting/utils.py`, which is shadowed by `reporting/utils/`
  - `SQLiteStateTracker`
  - `DISABLE_MEMORY_MANAGER = True`, which is hard-coded
  - five separate model-download implementations
- **ARCH-5: Layering runs the wrong way.**
  - The analysis code imports the GUI.
  - The workflow raises "GUI not available", so headless runs still need NiceGUI.

    - `import robin` loads torch, onnxruntime, sturgeon, pysam and the rest, for every CLI call and every Ray worker.
- **ARCH-6: File contracts are undocumented.**
    - Counter files are shared across modules (fusion and target use `cnv_analysis_counter.txt`, which is the cause of FUS-2).
    - Many writes are non-atomic, and pickles are used everywhere.
    - Coordinates, percent vs fraction, strand convention and which columns carry coverage are not written down anywhere.
- **ARCH-7: The GUI orchestrates the pipeline by polling files.** For example, a failed SNP job makes a bulk SNP queue wait up to 24 hours per sample.

---

11. Performance

ID	Problem	Measured / estimated	Where
PERF-1	Fusion GUI refresh runs an O(n^3) validation on the event loop every 30 s, and nothing reads the result	400 reads per gene pair took 21.5 s; 2,000 reads didn't finish in 10 min. The GUI freezes for all users	`gui/components/fusion.py:2736-2915, 1275-1487`
PERF-2	The MGMT timer runs methylartist inside the GUI process, on the event loop	Freezes on every MGMT batch	`gui/components/mgmt.py:550-603`
PERF-3	IGV code `time.sleep`-polls JavaScript calls that are never awaited	At least 5 s freeze on every details page load; status text is wrong	`gui/components/coverage.py:593-640, 744-1010`
PERF-4	The background scan holds a lock over all file I/O; a `ray.get(timeout=30)` runs inside an async handler	Multi-second GUI stalls with many samples	`gui_launcher.py:795-1092, 1384-1436`, `coverage.py:1639-1697`
PERF-5	The module-level `@ui.refreshable` rebuilds every client's summary	N^2 rebuilds: 5 tabs mean 75 rebuilds per 30 s	`gui/components/summary.py:64, 137, 241`
PERF-6	`stats()` waits behind busy pools	GUI stats freeze while a 30-60 min Clair3 job runs	`workflow_ray.py:3497-3498, 1309-1311, 3060-3066`
PERF-7	Each fusion batch rescans the whole-sample state and re-runs DBSCAN	O(N^2) over a run	`fusion_work.py:2675-2803, 4504-4559`
PERF-8	The per-BAM merge decodes binary columns row by row in Python, and round-trips through pandas and PyRanges	5.6 s of 11 s per 4M-row file; a vectorised `cast(pl.Utf8)` takes 0.04 s	`temp_utilities.py:223-266`, `matkit.py:992-1019`
PERF-9	BAMs are read many times (CNV 2-3×, target about 5×); ClinVar × regions is a pure-Python loop	CPU backlog at PromethION throughput	`cnv_analysis.py`, `target_analysis.py`, `lightweight_gene_analysis.py:560-612`
PERF-10	The report renders 24 per-chromosome figures at 300 dpi, then throws most away	Slow reports	`reporting/sections/cnv.py:988-1011`, `plotting.py`
PERF-11	MNP-Flex re-sends three base64 PNGs every 30 s	Constant traffic and flicker	`gui/components/mnpflex.py:722-765`

Suggested order of work

Make each step below its own change, with its own regression test. The scripts in `2026-10-16-repro/` are the starting point for those tests.

1. **Quick wins (each a few lines). Do SEC-1 first:**

- SEC-1: Ray dashboard on 127.0.0.1
- GUI-1: guard both loops in `_refresh_classification`
- REP-1: the RF config key
- JOB-6: change 60 to 3600, and make it configurable
- GUI-4: stop reading the counter file
- CNV-5: drop the last change point
- GUI-2: use `on_delete`
- VAR-3: force-rebuild the tabix index
- VAR-7: keep all panel regions

2. **CNV-1:** build the control profile without patient reads (see CNV-1; a deepcopy alone is not enough).
3. **The methylation maths, as one change with a test against `modkit pileup --combine-strands`:**
   - METH-1 and METH-2: aggregate always, and recompute % from counts
   - METH-5: semi-join, and dedupe the panel
   - METH-6: drop the `Start-1`
   - METH-3: matkit denominator
   - METH-4: combine strands
   - METH-7: pass coverage through
4. **Make failures visible:**
   - ERR-1: failures counted, and forwarded to the GUI
   - METH-8: classifiers raise instead of logging
   - VAR-4, REP-6: "not available" instead of a negative result
5. **Scheduling state:**
   - JOB-1: per-sample dirty flags
   - JOB-2: a persistent file ledger
   - CNV-2: no worker caches, and atomic writes
   - JOB-3, JOB-4, JOB-5, JOB-7
6. **Security hardening:** SEC-2 to SEC-6.
7. **Pin and record versions** (PKG-1), **add CI with a test suite** (PKG-2), then start on the architecture items. Delete dead code first (ARCH-4), because it is cheap and stops fixes from landing in the wrong copy.

Already done: the original "Error processing file ... ['Chromosome', 'Start']" log noise is fixed on `main` (commit `d02add2`), with a test. Commit `5be9e04` adds a traceback and safe diagnostics to that log line, because one clinical server still logs it after the fix.